



Contenido

01

¿Qué es una excepción en Java?

02

Diferencias claves y cuando usar cada tipo.

03

Excepciones más comunes en Java

04

Excepciones comunes

05

IOException : Excepciones Checked

06

Manejo de excepciones : Try-Catch-Finally

07

Throw vs throws en Java

08

Jerarquía de excepciones en Java

09

Excepciones propias en Java

10

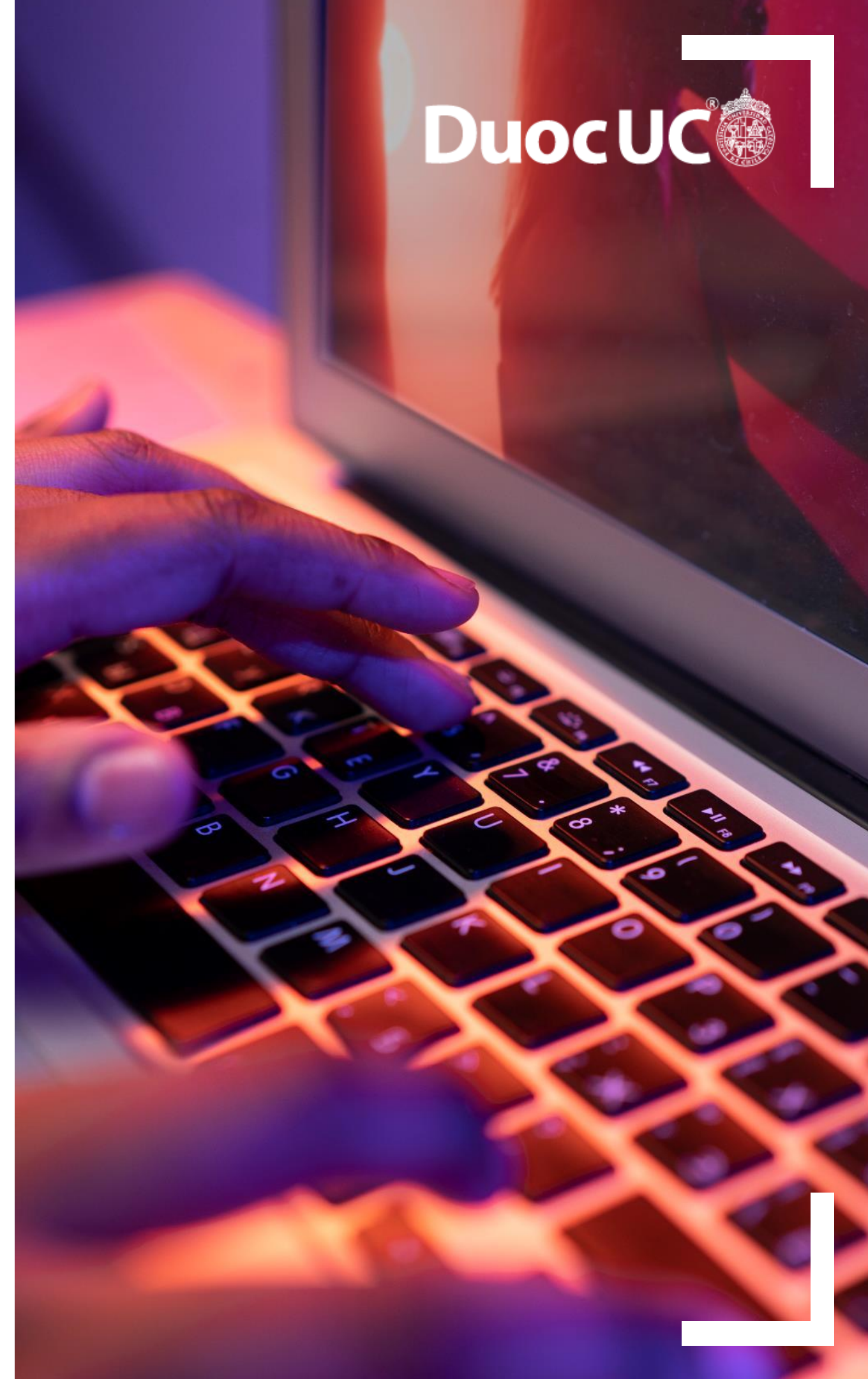
Ejemplo practico: Manejo de excepciones

11

Buenas prácticas y errores comunes.

12

Conclusión y próximo pasos



```
try { Animal perro = new Perro(); perro.hacerSonido(); } catch (NullPointerException e) { System.out.println("Error: " + e.getMessage()); }
```

</> Programación Orientada a Objetos

Java

Excepciones en Java

Manejo de errores, excepciones comunes y personalizadas con ejemplos de animales.



Errores Comunes
NullPointerException, Arithmetic



Manejo Seguro
try-catch-finally



ZooApp.java

```
// Ejemplo: acceso a animal nulo
Animal perro = null

// Lanza NullPointerException
try{
    perro.hacerSonido();
} catch (NullPointerException e) {
    System.out.println("Animal no inicializado");
}
```



⚠ ¿Qué es una excepción en Java?

Eventos que interrumpen el flujo normal del programa

💡 Concepto Central

Una **excepción** es un evento que ocurre durante la ejecución de un programa y que interrumpe el flujo normal de instrucciones.

⌚ **Interrumpe el flujo normal**
Detiene la ejecución secuencial del código.

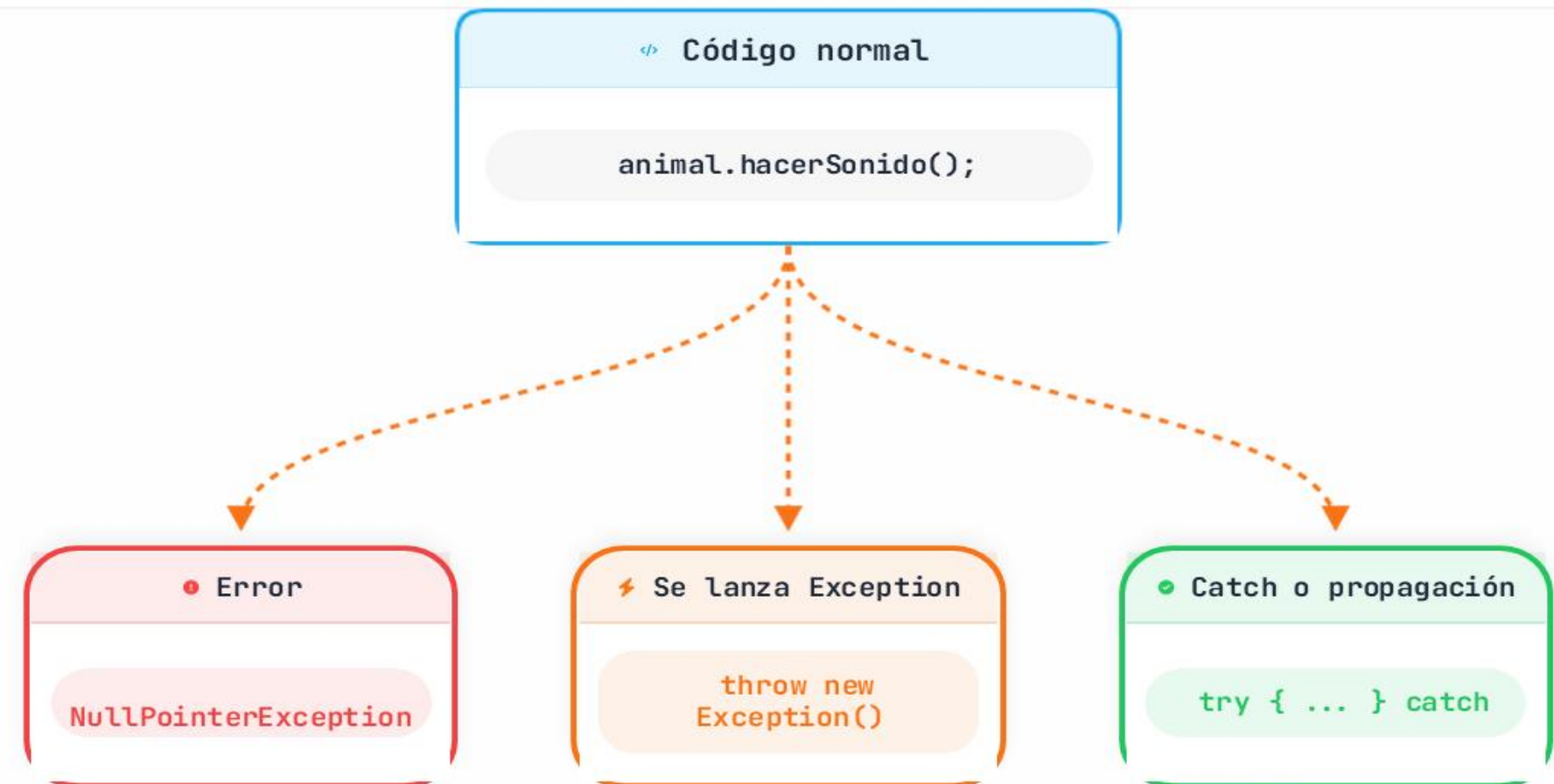
🚗 **Error recuperable o no**
Puede ser manejado o ser fatal.

👉 **Manejo explícito**
Permite try-catch o propagación.

En resumen:

- 🕒 Evento inesperado que altera el flujo.

Flujo de una excepción





Checked vs Unchecked Exceptions

Diferencias clave y cuándo usar cada tipo



Comparación de Tipos



Checked Exceptions

Deben capturarse o declararse

Ejemplo: IOException, SQLException

- ✓ Verificadas en tiempo de compilación
- ✓ Obligan a manejar o propagar



Unchecked Exceptions

RuntimeException y sus subclases

Ejemplo: NullPointerException, ArithmeticException

- ✓ Ocurren en tiempo de ejecución
- ✓ Errores de programación



Diferencia Clave

Checked: El compilador fuerza el manejo
Unchecked: No se verifican en compilación

</> Ejemplos con Animales



Checked IOException - Lectura de archivo

```
// Intentar leer archivo de animales
try {
    FileReader fr = new FileReader("animales.txt");
} catch (IOException e) {
    System.out.println("Error al leer");
}
```



Unchecked NullPointerException - Animal null

```
// Llamar método sobre animal null
Animal animal = null;
animal.hacerSonido(); // NPE
```

→ Manejo obligatorio vs. Opcional

try-catch

propagar



Manejo de Errores



Prevención de Errores





Excepciones más Comunes en Java

Checked y Unchecked Exceptions

⚠ Resumen

Las excepciones en Java se dividen en **checked** (deben ser capturadas o declaradas) y **unchecked** (RuntimeException).



NullPointerException

Acceder a un objeto null

animal.hacerSonido() cuando animal es null



ArithmeticException

División por cero

peso / cantidad cuando cantidad=0



ArrayIndexOutOfBoundsException

Índice fuera de rango

animales[5] en array de tamaño 3



NumberFormatException

Formato numérico inválido

Integer.parseInt("cachorro")

🗺 Mapa de Excepciones

● Runtime

● Checked

Runtime

NullPointerException

Ocurre al intentar acceder a un método o atributo de un objeto que es null.

Runtime

ArithmeticException

Ocurre en operaciones matemáticas inválidas como división por cero.

Checked

IOException

Ocurre en operaciones de entrada/salida como lectura de archivos.

// Ejemplo de manejo de excepciones

```
try {
    int resultado = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Error: " +
        e.getMessage());
} finally {
    System.out.println("Operación
    completada");
}
```

💡 Buena práctica

Siempre captura las excepciones específicas antes de las genéricas para un control más preciso.

⚠ Excepciones Runtime

✅ Excepciones Checked





Excepciones Comunes en Java

NullPointerException y ArithmeticException

❌ NullPointerException

Ocurre al intentar acceder a un objeto que es **null**.



Causa común

Llamar método sobre objeto null



Solución

Verificar null antes de acceder



ArithmeticException

Ocurre en operaciones aritméticas ilegales como división por cero.



Causa común

División por cero

</> Ejemplos con Animales



animal.hacerSonido() con null

```
Animal animal = null;  
animal.hacerSonido(); // ❌ NPE
```

Error: Objeto null

Fix: if (animal != null)



pesoPromedio / cantidad

```
int peso = 100;  
int cantidad = 0;  
int promedio = peso / cantidad;  
// ❌
```

Error: División por cero

Fix: if (cantidad != 0)

Flujo de Excepción

Runtime → Exception

60%

Prevención

Validación

80%



Manejo de Excepciones



⚠️ Excepciones Comunes

ArrayIndexOutOfBoundsException y NumberFormatException

🌟 Excepciones Más Comunes

Estas son las excepciones más frecuentes que ocurren al trabajar con arrays y conversiones de datos en Java.

📌 ArrayIndexOutOfBoundsException

Ocurre al intentar acceder a un índice que no existe en el array.

NumberFormatException

Ocurre al intentar convertir un String no numérico a número.

Consejo:

Siempre validar índices antes de acceder y usar try-catch para conversiones.

Ejemplos Prácticos con Animales

📄 Ejemplos de Excepciones

❗ ArrayIndexOutOfBoundsException

Intentando acceder a animales[5] en un array de tamaño 3

```
// Array con 3 animales
String[] animales = {"Perro", "Gato", "Pato"};
String animal = animales[5]; // ERROR
```

✅ **Solución:** Verificar animales.length

NumberFormatException

Intentando convertir "cachorro" a número entero

```
// Intentando parsear texto no numérico
String edadTexto = "cachorro";
int edad = Integer.parseInt(edadTexto); // ERROR
```

✅ **Solución:** Usar try-catch o matches("\\d+")



IOException: Excepciones Checked

Manejo de errores de E/S con archivos de animales

¿Qué es IOException?

IOException es una **excepción checked** que ocurre durante operaciones de entrada/salida. Como es checked, el compilador obliga a manejarla con try-catch o throws.



Archivo no existe

Intentar leer 'animales.txt' que no existe lanza FileNotFoundException.



Permisos denegados

No tener permisos para leer el archivo genera IOException.



Conexión perdida

Durante lectura de red, la conexión se interrumpe.

Importante:

- ✓ Siempre manejar o declarar excepciones checked

Flujo de Manejo de Errores



// Ejemplo de manejo de IOException

```
try {
  FileReader reader = new FileReader("animales.txt");
} catch (IOException e) {
  System.out.println("Error: " + e.getMessage());
}
```



Manejo de Excepciones: Try-Catch-Finally

Control de errores y liberación de recursos

Bloques Try-Catch-Finally

Estructura para manejar errores de forma controlada y garantizar la liberación de recursos.



try

Bloque donde se coloca el código que puede generar excepciones.



catch

Captura y maneja excepciones específicas lanzadas en try.

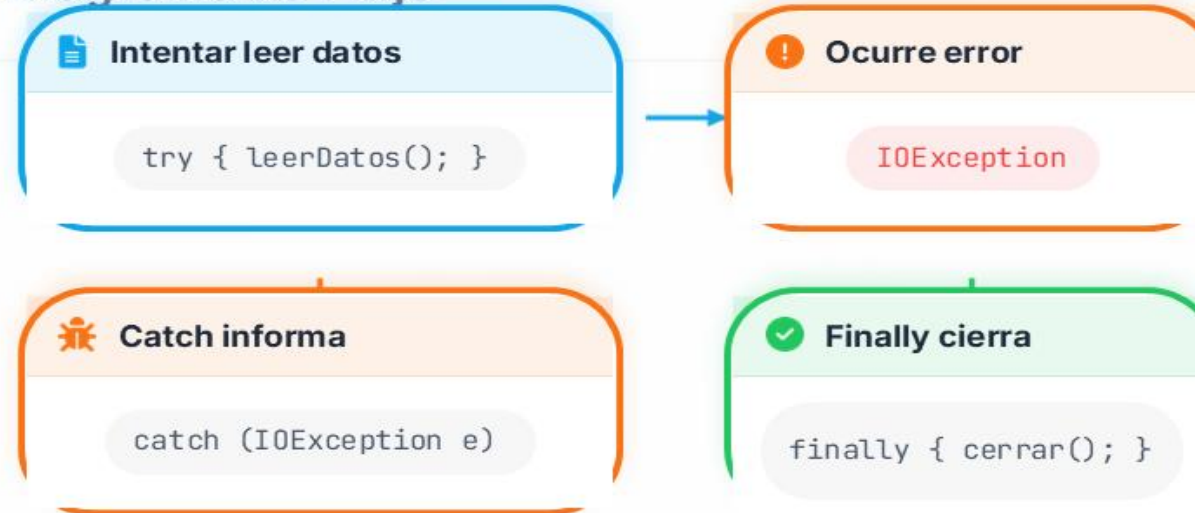


finally

Se ejecuta siempre, ideal para liberar recursos.

Flujo: try → error → catch → finally

Diagrama de Flujo



Ejemplo Java

```
try {  
    // Intentar leer archivo de animales  
    FileReader fr = new FileReader("animales.txt");  
} catch (IOException e) {  
    // Manejar error  
    System.out.println("Error: " + e.getMessage());  
} finally {  
    // Siempre ejecutar  
    cerrarRecursos();  
}
```



! throw vs throws en Java

Diferencias y uso correcto de excepciones con animales

Comparación Directa

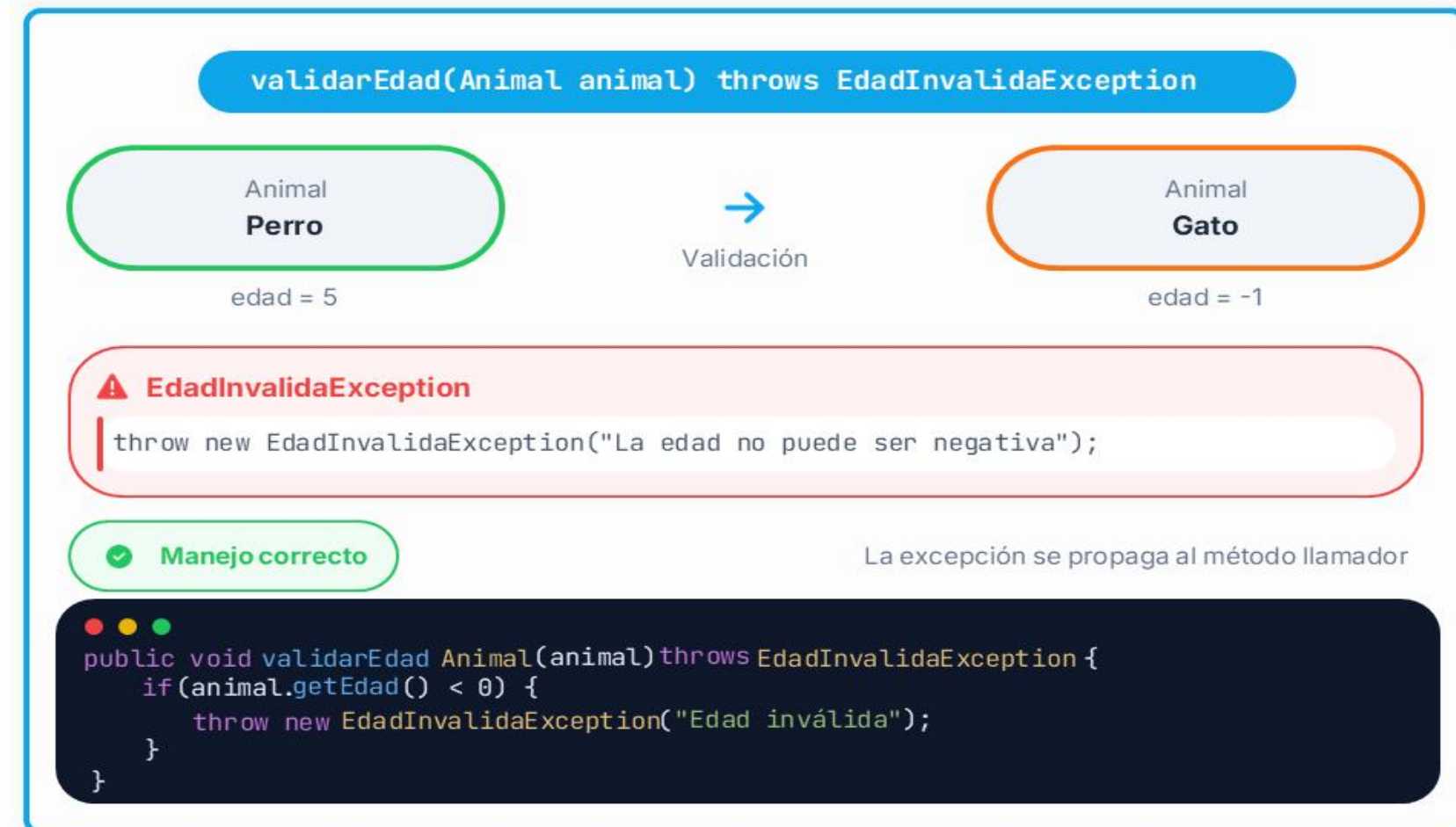
throw lanza una excepción manualmente, mientras que **throws** declara que un método puede lanzar excepciones.

Aspecto	throw	throws
Uso	Dentro del método	En la firma del método
Acción	Lanza la excepción	Declara la excepción
Cantidad	1 excepción por vez	Puede declarar múltiples
Control	Ejecución inmediata	Propaga al llamador

Regla de Oro:

💡 throw lanza, throws declara

Flujo de Excepciones



Jerarquía de Excepciones en Java

Checked vs Unchecked: Excepciones comunes en el zoológico

Estructura de Excepciones

Las excepciones en Java se organizan en una jerarquía donde **Exception** es la clase base.

Checked (Verificadas)

Deben declararse o capturarse obligatoriamente.

Unchecked (No verificadas)

No requieren declaración obligatoria.

Error

Errores graves del sistema.

En el zoológico:



Cada animal puede lanzar diferentes excepciones

Jerarquía Visual

Exception

Checked



IOException

Lectura de archivos de animales



SQLException

Conexión a base de datos

Unchecked



NullPointerException

Acceso a objeto null



ArithmeticException

División por cero



NumberFormatException

Formato numérico inválido



RuntimeException

! Excepciones Propias en Java

Creando excepciones personalizadas para tu sistema de zoológico

</> ¿Cuándo crear excepciones propias?

• Necesitas un tipo específico

La plataforma Java no tiene la excepción que necesitas

• Diferenciar errores

Los usuarios pueden identificar tu excepción de las de otros

• Múltiples excepciones relacionadas

Tu código lanza varias excepciones del mismo tipo

Ventaja clave:

Código más legible y manejo de errores más preciso

🏠 Jerarquía de Excepciones

Sistema de Zoológico

⚠️ **AnimalException** extends Exception

AnimalNoEncontradoException

Cuando el animal no existe

EdadInvalidaException

Cuando la edad es negativa

</> Ejemplo de uso en Zoológico

```
try {
    Animal animal = zoo.buscarAnimal("León");
    if (animal == null) {
        throw new AnimalNoEncontradoException("Animal no encontrado");
    }
} catch (AnimalNoEncontradoException e) {
    System.out.println("Error: " + e.getMessage());
}
```

Ejemplo Práctico: Manejo de Excepciones

Sistema de gestión de animales en un zoológico

Flujo del Proceso

1 Buscar Animal

Buscar animal por ID en la base de datos. Puede lanzar `AnimalNoEncontradoException`

2 Validar Edad

Verificar que la edad sea válida. Puede lanzar `IllegalArgumentException`

3 Registrar Animal

Guardar datos en archivo. Puede lanzar `IOException`

Resultado:

Animal registrado correctamente

Diagrama de Manejo de Excepciones



Buscar Animal

ID: 12345

● Encontrado



AnimalNoEncontrado

Cuando el ID no existe en la base de datos

```
throw new
AnimalNoEncontradoException(
"Animal no encontrado: " +
id);
```



IllegalArgumentException

Cuando la edad es negativa o inválida

```
throw new
IllegalArgumentException("Edad inválida: " + edad);
```



IOException

Cuando falla la escritura en archivo

```
throw new IOException("Error al escribir: " +
e.getMessage());
```



Manejo de Excepciones

Try-Catch

```
try { ... } catch (Exception e) { ... }
```

Finally

```
finally { cerrarRecursos(); }
```



Buenas Prácticas y Errores Comunes

Manejo efectivo de excepciones en Java

Errores a Evitar

-  **Capturar Exception genérica**
Evita `catch(Exception e)` sin filtrar
-  **Silenciar errores**
No usar `catch` vacío o `printStackTrace()`
-  **Mensajes poco claros**
Evitar "Error" sin contexto
-  **Lógica en catch**
No mezclar validación con manejo
-  **Excepciones como flujo normal**
No usar excepciones para control

 Estos errores dificultan el debugging

Checklist de Buenas Prácticas

-  **Usar excepciones específicas**
Preferir `NullPointerException` sobre `Exception`
-  **Mensajes descriptivos**
Incluir contexto en el mensaje
-  **Separar validación**
Validar antes de procesar
-  **Logging apropiado**
Registrar errores con detalles
-  **Cerrar recursos**
Usar `try-with-resources`

 Código más robusto y mantenible

Ejemplo: ZooApp

Validación de edad:

```
if (edad < 0 || edad > 150) {  
    throw new EdadInvalidaException(  
        "Edad no válida: " + edad);  
}
```

Manejo de recursos:

```
try (Scanner sc = new Scanner(file)) {  
    // procesar  
} catch (IOException e) {  
    logger.error("Error: " + e);  
}
```

 **Resultado:** Código limpio y seguro

 **Consejo:** Usa excepciones para casos excepcionales, no para control de flujo normal.



Conclusiones y Próximos Pasos

Cierre de Excepciones en Java

Síntesis Final

El manejo de **excepciones** permite crear aplicaciones robustas que se recuperan de errores, proporcionan diagnósticos claros y mantienen un código limpio.



Robustez

Aplicaciones que se recuperan de errores inesperados.



Diagnóstico claro

Mensajes informativos para depuración rápida.



Código mantenible

Separación clara entre lógica y manejo de errores.

Resultado:

 Aplicaciones más seguras y resilientes

Visión General del Manejo de Excepciones



→ Próximos Pasos

1

Practicar checked/unchecked

2

Crear excepciones propias

3

Mejorar logs

Felicitaciones
¡Has finalizado el módulo!

DuocUC[®]



CERCANÍA. LIDERAZGO. FUTURO.

7
AÑOS
ACREDITADO

 Comisión Nacional
de Acreditación
CNA-Chile

NIVEL DE EXCELENCIA
HASTA OCTUBRE 2031

Docencia de pregrado / Gestión institucional / Aseguramiento interno de la
calidad / Vinculación con el Medio / Investigación, creación y/o innovación